Rapport detaille : Jenkins avec Docker

Installation, configuration, integration CI/CD du projet **fullStack_portfolio**, role de chaque element et exemples pratiques.

Projet
Portfolio React/Vite + API Express + MongoDB

Date de realisation
15 mai 2026

Environnement
Windows + PowerShell + Docker Desktop

Resultat
Jenkins actif sur http://localhost:8080

1. Objectif du travail

L'objectif etait de mettre en place Jenkins pour automatiser le cycle CI/CD du projet fil rouge. Le projet contient un frontend React/Vite, une API Express, une connexion MongoDB et des scripts npm. Jenkins doit pouvoir recuperer le code, installer les dependances, verifier le projet, construire le frontend, archiver le resultat et preparer une image Docker.

Docker a ete utilise pour installer Jenkins rapidement sans modifier l'installation Windows. Jenkins tourne donc dans un conteneur, avec ses donnees conservees dans un volume Docker.

2. Resume tres court du resultat final

Element	Resultat obtenu	A quoi ca sert
Jenkins	Conteneur Docker lance, accessible sur `http://localhost:8080`	Interface CI/CD pour executer les builds automatiquement.
Volume Docker	`jenkins_home`	Conserver les donnees Jenkins meme si le conteneur est supprime.
Reseau Docker	`jenkins`	Isoler et organiser les conteneurs Jenkins.
Jenkinsfile	Ajoute a la racine du projet	Decrire le pipeline Jenkins sous forme de code.
Dockerfile du projet	Image `fullstack-portfolio:jenkins-demo` construite	Preparer une version deployable du projet.
Documentation	`docs/jenkins.md` et ce rapport PDF	Expliquer l'installation, la demo et l'integration.

3. Concepts importants avant les commandes

Concept	Explication simple	Exemple dans ce projet
Docker	Plateforme qui execute des applications dans des conteneurs isoles.	Jenkins est execute dans un conteneur au lieu d'etre installe directement sur Windows.
Image Docker	Modele de base qui contient tout ce qu'il faut pour lancer une application.	`jenkins/jenkins:lts-jdk21` est l'image officielle utilisee pour Jenkins.
Conteneur	Instance en execution d'une image Docker.	Le conteneur s'appelle `jenkins`.
Volume	Stockage persistant gere par Docker.	`jenkins_home` conserve les jobs, plugins, utilisateurs et configurations Jenkins.
Reseau Docker	Reseau virtuel qui permet aux conteneurs de communiquer.	`jenkins` est le reseau cree pour organiser Jenkins.
Port mapping	Association entre un port du PC et un port du conteneur.	`-p 8080:8080` permet d'ouvrir Jenkins depuis le navigateur.
CI/CD	Automatisation de l'integration, des tests, du build et du deploiement.	Jenkins execute `npm ci`, `npm run build` et archive `dist/`.

4. Verification de Docker Desktop

Avant d'installer Jenkins, il fallait verifier que Docker Desktop etait lance et que le moteur Docker repondait. Sans le moteur Docker, la commande `docker run` ne peut pas creer le conteneur Jenkins.

```
docker info
```

A quoi ca sert : afficher l'etat du client Docker et du serveur Docker.

Exemple de resultat utile :

```
Server:
 Containers: 3
 Server Version: 29.4.2
 Operating System: Docker Desktop
 OSType: linux
 Architecture: x86_64
```

> La verification a confirme que Docker Desktop etait actif. Jenkins pouvait donc etre installe avec Docker.

5. Verification d'une ancienne installation Jenkins

Avant de creer un nouveau conteneur, j'ai verifie s'il existait deja un conteneur, un volume ou un reseau Jenkins. Cela evite d'ecraser une installation existante.

```
docker ps -a --filter name=jenkins --format "{{.Names}}\t{{.Status}}\t{{.Ports}}"
docker volume ls --filter name=jenkins_home --format "{{.Name}}"
```

```
docker network ls --filter name=jenkins --format "{{.Name}}"
```

Commande	A quoi ca sert	Exemple
`docker ps -a --filter name=jenkins`	Voir si un conteneur Jenkins existe deja, meme s'il est arrete.	Si rien ne s'affiche, aucun conteneur Jenkins n'existe.
`docker volume ls --filter name=jenkins_home`	Voir si un volume Jenkins existe deja.	Si `jenkins_home` existe, les donnees Jenkins sont deja presentes.
`docker network ls --filter name=jenkins`	Voir si le reseau Jenkins existe deja.	Si rien ne s'affiche, on peut creer le reseau.

Dans ce cas, aucune ancienne installation Jenkins n'a ete detectee. L'installation pouvait donc commencer proprement.

6. Creation du reseau Docker Jenkins

```
docker network create jenkins
```

A quoi ca sert : creer un reseau Docker nomme `jenkins`.

Meme si Jenkins peut fonctionner avec le reseau Docker par defaut, creer un reseau dedie rend l'installation plus propre. Plus tard, si on ajoute un agent Jenkins, un conteneur Docker-in-Docker ou un autre service, on pourra les connecter au meme reseau.

Exemple concret : un agent Jenkins pourrait rejoindre ce reseau pour communiquer avec le serveur Jenkins.

```
docker run --network jenkins mon-agent-jenkins
```

7. Creation du volume Jenkins

```
docker volume create jenkins_home
```

A quoi ca sert : creer un stockage persistant pour Jenkins.

Jenkins stocke ses donnees dans `/var/jenkins_home` a l'interieur du conteneur. Si ces donnees restent uniquement dans le conteneur, elles peuvent etre perdues en cas de suppression du conteneur. Le volume `jenkins_home` evite ce probleme.

Sans volume	Avec volume
Suppression du conteneur = risque de perdre la configuration Jenkins.	Suppression du conteneur = les donnees restent dans `jenkins_home`.

Exemple : les plugins installes, les jobs crees et les utilisateurs Jenkins sont gardes dans ce volume.

8. Installation et lancement de Jenkins

```
docker run --name jenkins -d `
  --restart=on-failure `
  --network jenkins `
  -p 8080:8080 -p 50000:50000 `
  -v jenkins_home:/var/jenkins_home `
  jenkins/jenkins:lts-jdk21
```

Cette commande est le coeur de l'installation Jenkins.

Partie de la commande	A quoi ca sert	Exemple ou effet
`docker run`	Creer et demarrer un conteneur depuis une image Docker.	Docker cree le conteneur Jenkins.
`--name jenkins`	Donner un nom simple au conteneur.	On peut ensuite utiliser `docker logs jenkins`.
`-d`	Lancer le conteneur en arriere-plan.	Le terminal reste disponible pendant que Jenkins tourne.
`--restart=on-failure`	Relancer Jenkins automatiquement si le conteneur s'arrete a cause d'une erreur.	Utile pour garder Jenkins disponible apres un crash.
`--network jenkins`	Connecter le conteneur au reseau Docker cree plus haut.	Jenkins appartient au reseau `jenkins`.
`-p 8080:8080`	Exposer l'interface web Jenkins.	Acces navigateur : `http://localhost:8080`.
`-p 50000:50000`	Exposer le port utilise par certains agents Jenkins.	Utile si Jenkins pilote des agents externes.
`-v jenkins_home:/var/jenkins_home`	Connecter le volume persistant au dossier Jenkins.	Les donnees Jenkins restent sauvegardees.
`jenkins/jenkins:lts-jdk21`	Image Jenkins officielle LTS avec Java 21.	Docker la telecharge si elle n'existe pas localement.

Resultat : Docker a telecharge l'image Jenkins puis a demarre le conteneur.

9. Verification du conteneur Jenkins

```
docker ps --filter name=jenkins --format "{{.Names}}\t{{.Status}}\t{{.Ports}}"
```

A quoi ca sert : verifier que le conteneur Jenkins tourne et que les ports sont ouverts.

Exemple de resultat obtenu :

```
jenkins    Up 2 minutes    0.0.0.0:8080->8080/tcp, 0.0.0.0:50000->50000/tcp
```

Cette ligne signifie que Jenkins est en marche et que le port 8080 du PC redirige bien vers le port 8080 du conteneur.

10. Lecture des logs Jenkins

```
docker logs --tail 80 jenkins
```

A quoi ca sert : voir les messages de demarrage Jenkins et recuperer les informations importantes.

Les logs indiquent que Jenkins demarre, charge ses plugins internes, prepare l'assistant d'installation et genere un mot de passe initial.

```
Jenkins initial setup is required.
Please use the following password to proceed to installation:
********
This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
```

> Le mot de passe initial ne doit pas etre publie dans un rapport public. Il sert uniquement a debloquer Jenkins lors de la premiere connexion.

11. Recuperation du mot de passe initial Jenkins

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

A quoi ca sert : executer une commande a l'interieur du conteneur Jenkins pour lire le fichier du mot de passe initial.

Partie	Explication
`docker exec`	Execute une commande dans un conteneur deja lance.
`jenkins`	Nom du conteneur cible.
`cat /var/jenkins_home/secrets/initialAdminPassword`	Affiche le contenu du fichier contenant le mot de passe initial.

Exemple d'utilisation : copier ce mot de passe, ouvrir Jenkins dans le navigateur, puis le coller dans l'ecran "Unlock Jenkins".

12. Verification de l'interface web Jenkins

```
Invoke-WebRequest -UseBasicParsing http://localhost:8080/login -TimeoutSec 20
```

A quoi ca sert : verifier depuis PowerShell que l'interface Jenkins repond.

Resultat obtenu :

```
200
```

Le code HTTP `200` signifie que la page existe et que Jenkins repond correctement. L'interface est donc accessible dans le navigateur.

13. Premiere configuration a faire dans Jenkins

Apres l'installation Docker, il faut finir l'assistant Jenkins dans le navigateur.

1. Ouvrir `http://localhost:8080`.
2. Coller le mot de passe initial recupere avec `docker exec`.
3. Choisir **Install suggested plugins**.
4. Creer un utilisateur administrateur.
5. Valider l'URL Jenkins proposee : `http://localhost:8080/`.
6. Arriver sur le tableau de bord Jenkins.

Etape dans l'interface	A quoi ca sert	Exemple
Unlock Jenkins	Verifier que l'utilisateur a acces a l'installation Jenkins.	Coller le mot de passe initial.
Install suggested plugins	Installer les plugins de base pour creer des jobs et pipelines.	Git, Pipeline, credentials, interface web.
Create First Admin User	Creer le compte qui administrera Jenkins.	Identifiant, mot de passe, email.
Jenkins URL	Definir l'adresse principale de Jenkins.	`http://localhost:8080/`.

14. Plugins Jenkins utiles pour ce projet

Plugin	A quoi ca sert	Exemple dans le projet
Pipeline	Permet d'executer un pipeline defini dans un `Jenkinsfile`.	Executer les stages checkout, install, build, smoke check.
Git	Permet a Jenkins de cloner un depot Git.	Recuperer le projet `fullStack_portfolio`.
GitHub Branch Source	Detecte branches et pull requests GitHub.	Creer automatiquement des builds par branche.
NodeJS	Installe ou selectionne une version Node.js pour les jobs.	Executer `npm ci` et `npm run build`.
Docker Pipeline	Facilite l'utilisation de Docker dans les pipelines.	Construire une image Docker depuis Jenkins.
Credentials Binding	Injecte des secrets dans un job sans les ecrire dans le code.	Mot de passe Docker Hub, token GitHub, variables de production.
Email Extension	Envoie des emails personnalises selon le resultat du build.	Notifier l'equipe si le build echoue.

15. Creation du Jenkinsfile

J'ai ajoute un fichier `Jenkinsfile` a la racine du projet. Ce fichier decrit tout ce que Jenkins doit faire. C'est ce qu'on appelle un pipeline as code.

A quoi ca sert : le pipeline devient versionne avec le projet. Si le projet est clone ailleurs, Jenkins sait toujours comment le construire.

```
pipeline {
    agent any
    stages {
        stage("Checkout") {
            steps {
                checkout scm
            }
        }
        stage("Install dependencies") {
            steps {
                script {
                    runCommand("npm ci")
                }
            }
        }
        stage("Build frontend") {
            steps {
                script {
                    runCommand("npm run build")
                }
            }
        }
    }
}
```

Exemple : quand on clique sur `Build Now`, Jenkins lit ce fichier et execute les etapes dans l'ordre.

16. Detail des stages Jenkins ajoutes

Stage	A quoi ca sert	Commande ou exemple
Checkout	Recuperer le code source depuis Git.	`checkout scm`
Install dependencies	Installer les dependances exactes du projet depuis `package-lock.json`.	`npm ci`
Quality checks	Lancer les tests si un script de test existe.	`npm run test --if-present`
Build frontend	Construire le frontend React/Vite.	`npm run build`
Smoke check	Verifier que le build a produit `dist/index.html`.	`node -e "const fs=require('fs'); ..."`
Docker image	Construire l'image Docker du projet si le parametre est coche.	`docker build -t fullstack-portfolio:BUILD_NUMBER .`
Post actions	Archiver les artefacts et envoyer les notifications.	`archiveArtifacts artifacts: "dist/**"`

17. Pourquoi j'ai utilise npm ci dans Jenkins

`npm ci` est plus adapte a la CI que `npm install`. Il installe les dependances a partir de `package-lock.json` et donne un environnement plus reproductible.

Commande	Utilisation	Exemple
`npm install`	Pratique en developpement, peut modifier le lockfile.	Ajouter une dependance localement.
`npm ci`	Pratique en CI, installation propre et reproductible.	Utilise dans Jenkins.

18. Build Vite du projet

```
npm.cmd run build
```

A quoi ca sert : generer une version production du frontend React.

Exemple de resultat :

```
vite v8.0.8 building client environment for production...
dist/index.html
dist/assets/index-....css
dist/assets/index-....js
✓ built
```

Le dossier `dist/` est l'artefact principal du frontend. Jenkins l'archive pour permettre de le telecharger depuis l'interface du build.

19. Adaptation de Express pour servir dist en production

Le projet avait deja une API Express. J'ai adapte `app.js` pour que, en production, Express serve aussi les fichiers generes par Vite dans `dist/`.

```
if (process.env.NODE_ENV === "production") {
    app.use(express.static(distPath));
    app.get(/^(?!\/api).*/, (_req, res) => {
        return res.sendFile(path.join(distPath, "index.html"));
    });
}
```

Element	A quoi ca sert	Exemple
`express.static(distPath)`	Servir les fichiers HTML, CSS et JS generes.	`/assets/index.js` est envoye au navigateur.
`app.get(/^(?!\/api).*/)`	Rediriger les routes React vers `index.html`.	`/portfolio` affiche l'application React.
`NODE_ENV=production`	Activer le comportement de production.	Utilise dans le Dockerfile runtime.

20. Creation du Dockerfile du projet

J'ai ajoute un `Dockerfile` multi-stage. Cela permet de construire le frontend dans une premiere etape, puis de garder une image finale plus propre pour l'execution.

```
FROM node:22-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:22-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY app.js ./
COPY config ./config
COPY controllers ./controllers
COPY lib ./lib
COPY middleware ./middleware
COPY models ./models
COPY public ./public
COPY routes ./routes
COPY --from=build /app/dist ./dist
EXPOSE 5000
CMD ["node", "app.js"]
```

Ligne	A quoi ca sert	Exemple
`FROM node:22-alpine AS build`	Utiliser Node.js pour construire le projet.	Vite a besoin de Node pour generer `dist/`.
`RUN npm ci`	Installer toutes les dependances, y compris les dependances de build.	Installer Vite et React.
`RUN npm run build`	Construire le frontend.	Creer `dist/index.html`.
`FROM node:22-alpine AS runtime`	Creer l'image finale d'execution.	Elle contient l'API et le build final.
`npm ci --omit=dev`	Installer seulement les dependances necessaires en production.	Express, mongoose, dotenv, cors.
`COPY --from=build /app/dist ./dist`	Copier le build frontend dans l'image finale.	Express peut servir `dist/`.
`EXPOSE 5000`	Documenter le port utilise par l'application.	L'application Express ecoute sur 5000.
`CMD ["node", "app.js"]`	Definir la commande lancee au demarrage du conteneur.	Demarrer l'API Express.

21. Creation du .dockerignore

Le fichier `.dockerignore` evite d'envoyer des fichiers inutiles ou sensibles au contexte de build Docker.

```
node_modules
dist
.git
.env
.env.*
!.env.example
coverage
*.log
.vite
.cache
.vscode
.idea
```

Element ignore	Pourquoi	Exemple
`node_modules`	Les dependances sont reinstallees proprement dans Docker.	Evite de copier des milliers de fichiers locaux.
`.env`	Contient des secrets ou variables locales.	`MONGO_URI`, `ADMIN_SECRET`.
`dist`	Le build est regenere dans Docker.	Evite un ancien build non controle.
`.git`	L'historique Git n'est pas necessaire dans l'image.	Image plus legere.

22. Construction de l'image Docker du projet

```
docker build -t fullstack-portfolio:jenkins-demo .
```

A quoi ca sert : verifier que le projet peut etre transforme en image Docker de production.

Partie	Explication	Exemple
`docker build`	Construit une image Docker depuis le Dockerfile.	Docker lit les instructions du fichier.
`-t fullstack-portfolio:jenkins-demo`	Donne un nom et un tag a l'image.	Nom : `fullstack-portfolio`, tag : `jenkins-demo`.
`.`	Utilise le dossier courant comme contexte Docker.	Le projet actuel est envoye a Docker.

Resultat obtenu : l'image `fullstack-portfolio:jenkins-demo` a ete construite correctement.

23. Verification des images Docker

```
docker images --format "{{.Repository}}:{{.Tag}}\t{{.ID}}\t{{.Size}}"
```

A quoi ca sert : afficher les images disponibles localement.

Exemples presents apres le travail :

```
jenkins/jenkins:lts-jdk21        799MB
fullstack-portfolio:jenkins-demo 422MB
```

La premiere image sert a lancer Jenkins. La deuxieme image correspond au projet fil rouge pret a etre execute en conteneur.

24. Exemple pour lancer le projet en Docker

Une fois l'image construite, on peut lancer l'application avec les variables necessaires.

```
docker run --rm -p 5000:5000 `
  -e MONGO_URI="mongodb://host.docker.internal:27017/fullstack_portfolio" `
  -e ADMIN_PASSWORD="mot-de-passe-fort" `
  -e ADMIN_SECRET="cle-longue-et-secrete" `
  fullstack-portfolio:jenkins-demo
```

Variable	A quoi ca sert	Exemple
`MONGO_URI`	Adresse de la base MongoDB.	`mongodb://host.docker.internal:27017/fullstack_portfolio`
`ADMIN_PASSWORD`	Mot de passe admin de l'application portfolio.	`mot-de-passe-fort`
`ADMIN_SECRET`	Secret utilise pour verifier le token admin.	`cle-longue-et-secrete`

25. Creation d'un job Pipeline Jenkins pour ce projet

Apres l'assistant initial, le job Jenkins peut etre cree depuis l'interface.

1. Cliquer sur **New Item**.
2. Nommer le job `fullstack-portfolio`.
3. Choisir **Pipeline**.
4. Dans **Pipeline**, choisir **Pipeline script from SCM**.
5. Choisir **Git**.
6. Coller l'URL du depot Git.
7. Ajouter les credentials si le depot est prive.
8. Verifier que **Script Path** vaut `Jenkinsfile`.
9. Cliquer sur **Save**.
10. Cliquer sur **Build Now**.

Exemple : si le depot est public, Jenkins peut cloner directement l'URL HTTPS du depot. S'il est prive, il faut ajouter un token ou une cle SSH dans **Manage Jenkins > Credentials**.

26. Gestion des credentials Jenkins

Les credentials permettent de garder les secrets hors du code source. Il ne faut pas ecrire les mots de passe dans le `Jenkinsfile`.

Secret	A quoi ca sert	Exemple d'utilisation
Token GitHub	Cloner un depot prive.	Credential de type username/password ou secret text.
Mot de passe Docker Hub	Pousser une image vers Docker Hub.	`docker login` dans Jenkins avec credential binding.
`MONGO_URI`	Connecter l'API a une base distante.	Injecter la variable au moment du deploiement.
`ADMIN_SECRET`	Securiser les routes admin.	Secret text Jenkins.

27. Notifications Jenkins

Le `Jenkinsfile` contient un parametre `EMAIL_RECIPIENTS`. Si SMTP et le plugin Email Extension sont configures, Jenkins peut envoyer un email apres le build.

```
parameters {
    string(
        name: "EMAIL_RECIPIENTS",
        defaultValue: "",
        description: "Optional comma-separated email list for Jenkins notifications."
    )
}
```

A quoi ca sert : informer automatiquement l'equipe si un build reussit ou echoue.

Exemple : mettre `dev@example.com,admin@example.com` dans le parametre du build.

28. Artefacts Jenkins

```
archiveArtifacts artifacts: "dist/**", fingerprint: true, allowEmptyArchive: true
```

A quoi ca sert : sauvegarder le resultat du build dans Jenkins.

Ici, l'artefact principal est le dossier `dist/`, qui contient le frontend compile. Apres un build, on peut le telecharger depuis la page du build Jenkins.

Exemple : apres `npm run build`, Jenkins archive `dist/index.html` et les fichiers CSS/JS generes.

29. Comparaison entre Jenkins et GitHub Actions

Point	Jenkins	GitHub Actions
Installation	Serveur a installer et administrer.	Integre directement a GitHub.
Controle	Tres fort controle sur l'environnement.	Controle plus simple, surtout avec runners heberges.
Plugins	Grand ecosysteme de plugins.	Actions reutilisables depuis le marketplace.
Cas d'usage	Environnements internes, CI/CD personnalisee, entreprises.	Projets GitHub, CI rapide, workflows cloud.

30. Commandes utiles apres installation

Commande	A quoi ca sert	Exemple
`docker ps`	Voir les conteneurs en cours d'execution.	Verifier que Jenkins est `Up`.
`docker stop jenkins`	Arreter Jenkins.	Utile avant d'eteindre la machine.
`docker start jenkins`	Redemarrer Jenkins.	Relancer Jenkins le lendemain.
`docker logs jenkins`	Voir les logs Jenkins.	Diagnostiquer un probleme de demarrage.
`docker exec -it jenkins bash`	Ouvrir un shell dans le conteneur Jenkins.	Inspecter `/var/jenkins_home`.
`docker volume inspect jenkins_home`	Voir les details du volume Jenkins.	Confirmer que les donnees sont persistantes.

31. Problemes possibles et solutions

Probleme	Cause possible	Solution
`Cannot connect to Docker daemon`	Docker Desktop n'est pas lance.	Ouvrir Docker Desktop puis relancer `docker info`.
Port 8080 deja utilise	Une autre application utilise le port.	Lancer Jenkins avec `-p 8081:8080`.
Mot de passe initial introuvable	Jenkins est deja configure ou le fichier a ete supprime.	Verifier les logs ou utiliser le compte admin cree.
Pipeline echoue sur `npm ci`	Node.js absent sur l'agent Jenkins ou lockfile incoherent.	Installer/configurer NodeJS dans Jenkins ou corriger `package-lock.json`.
Stage Docker echoue dans Jenkins	Docker n'est pas accessible depuis l'agent Jenkins.	Installer Docker sur l'agent ou configurer un agent dedie.

32. Ce qui a permis de realiser le travail

- **Docker Desktop** : a permis de lancer Jenkins dans un conteneur local.
- **Image Jenkins officielle** : a fourni Jenkins pret a demarrer avec Java 21.
- **Volume jenkins_home** : a garde les donnees Jenkins persistantes.
- **Projet Node/Vite** : disposait deja d'un script `npm run build`.
- **Jenkinsfile** : a transforme les etapes CI/CD en code versionne.
- **Dockerfile** : a permis de construire une image de production du projet.
- **.dockerignore** : a protege les fichiers locaux et allege le build Docker.
- **app.js adapte** : a permis a Express de servir le frontend compile.

33. Scenario final de demonstration

1. Montrer Docker Desktop lance.
2. Executer `docker ps` pour montrer le conteneur Jenkins actif.
3. Ouvrir `http://localhost:8080`.
4. Expliquer le volume `jenkins_home`.
5. Expliquer le reseau `jenkins`.
6. Ouvrir le projet et montrer `Jenkinsfile`.
7. Creer ou ouvrir le job `fullstack-portfolio`.
8. Lancer **Build Now**.
9. Montrer les stages du pipeline.
10. Montrer les artefacts `dist/**`.
11. Activer le parametre Docker si l'agent Jenkins a acces a Docker.
12. Montrer l'image `fullstack-portfolio:jenkins-demo`.

34. Conclusion

Jenkins est maintenant installe avec Docker et le projet contient tous les fichiers necessaires pour une integration CI/CD. Le pipeline peut installer les dependances, executer les controles disponibles, construire le frontend, verifier le resultat, archiver les fichiers generes et construire une image Docker.

Cette approche est adaptee a une demonstration pedagogique parce qu'elle montre a la fois Jenkins, Docker, Git, Node.js, les artefacts, les plugins, les notifications et une strategie de deploiement possible.

Document genere localement depuis le projet fullStack_portfolio.